

Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería Y
Agrimensura



Análisis de Lenguajes de Programación R322

Trabajo Práctico Final - Informe

Alumnos:
Julián Paz

Índice general

1. Introducción	1
1.1. Bases de datos relacionales	1
1.2. Álgebra Relacional	2
2. Intérprete del Álgebra Relacional	3
2.1. Definición de una base de datos relacional	3
2.2. Consultas sobre una base de datos relacional	3
2.3. Definición de operadores personalizados	4
2.4. Sintaxis del EDSL	4
2.5. Ejemplos	8
2.6. Detalles del diseño del EDSL	17
3. Implementación del EDSL	19
3.1. Módulos del trabajo	19

1 Introducción

Comencemos viendo una breve introducción a las bases de datos relacionales y al álgebra relacional.

1.1. Bases de datos relacionales

Una base de datos relacional consiste en un conjunto de tablas, a cada una de las cuales se le asigna un nombre exclusivo. Cada fila de la tabla representa una *relación* entre un conjunto de valores. De manera informal, cada tabla es un conjunto de entidades, y cada fila es una entidad.

Considérese la siguiente tabla (a la cual llamaremos tabla **cuenta**):

número_cuenta	nombre_sucursal	saldo
C-101	Centro	500
C-102	Navacerrada	400
C-201	Galapagar	900
C-215	Becerril	700
C-217	Galapagar	750
C-222	Moralzarzal	700
C-305	Collado Mediano	350

Tiene tres cabeceras de columna:

- **número_cuenta.**
- **nombre_sucursal.**
- **saldo.**

Siguiendo la terminología del modelo relacional, se puede hacer referencia a estas cabeceras como *atributos*.

Para cada atributo hay un conjunto de valores permitidos, denominado *dominio* de ese atributo. Por ejemplo el dominio del atributo **nombre_sucursal** es el conjunto de todos los nombres de sucursal.

Ahora supóngase que D_1 denota el conjunto de todos los números de cuenta, D_2 el conjunto de todos los nombres de sucursal y D_3 el conjunto de todos los saldos. Todas las filas de la tabla cuenta deben consistir en una tupla (v_1, v_2, v_3) , donde v_1 es un número de cuenta (es decir, $v_1 \in D_1$), v_2 es un nombre de sucursal ($v_2 \in D_2$) y v_3 es un saldo ($v_3 \in D_3$). En general, cuenta sólo contendrá un subconjunto del conjunto de todas las filas posibles. Por tanto, la tabla cuenta será un subconjunto de:

$$D_1 \times D_2 \times D_3$$

En general, una tabla de n atributos debe ser un subconjunto de:

$$D_1 \times D_2 \times \cdots \times D_{n-1} \times D_n$$

Los matemáticos definen las relaciones como subconjuntos del producto cartesiano de la lista de dominios. Esta definición se corresponde de manera casi exacta con la definición de tabla dada anteriormente. La única diferencia es que aquí se han asignado nombres a los atributos, mientras que los matemáticos sólo usan "nombres" numéricos, usando el entero 1 para denotar el atributo cuyo dominio aparece en primer lugar de la lista de dominios, 2 para el atributo cuyo dominio aparece en segundo lugar, etc.

Como las tablas son, esencialmente, relaciones, se usarán los términos matemáticos *relación* y *tupla* en lugar de los términos *tabla* y *fila*.

Un valor de dominio que es miembro de todos los dominios posibles es el valor *nulo*, que indica que el valor es desconocido o no existe.

Cuando se habla de bases de datos se debe diferenciar entre el *esquema de la base de datos*, que es el diseño lógico de la misma, y el *ejemplar de la base de datos*, que es una instantánea de los datos de la misma en un momento dado. Por ejemplo el esquema de la relación **cuenta** es el siguiente:

$$\text{Esquema_cuenta}=(\text{nombre_cuenta}, \text{nombre_sucursal}, \text{saldo})$$

En general, los esquemas de las relaciones consisten en una lista de los atributos y de sus dominios correspondientes.

1.2. Álgebra Relacional

Un lenguaje de consultas es un lenguaje en el que los usuarios solicitan información de la base de datos. Los lenguajes de consultas pueden clasificarse como:

- Procedimentales: En esta clase de lenguajes el usuario indica al sistema que lleve a cabo una serie de operaciones en la base de datos para calcular el resultado deseado
- No procedimentales: En los lenguajes de consulta no procedimentales el usuario describe la información deseada sin dar un procedimiento concreto para obtener esa información.

El álgebra relacional es un lenguaje de consulta procedimental que forma la base teórica de los sistemas de bases de datos relacionales.

El álgebra relacional consiste en un conjunto de operaciones que toman una o dos relaciones como entrada y generan otra relación nueva como resultado. Las operaciones fundamentales del álgebra relacional son *selección*, *proyección*, *unión*, *diferencia de conjuntos*, *producto cartesiano* y *renombramiento*.

2 Intérprete del Álgebra Relacional

Este trabajo implementa un intérprete para un EDSL (Embedded Domain Specific Language) basado en álgebra relacional. El objetivo es crear una herramienta que permita:

1. Definir una base de datos relacional.
2. Realizar consultas del álgebra relacional sobre una base de datos relacional.
3. Definir operadores personalizados para extender el lenguaje

En esta sección vamos a ver como es que se implementan todas estas funcionalidades.

2.1. Definición de una base de datos relacional

Para poder definir una base de datos relacional es necesario tener un DDL (Data Definition Language). El álgebra relacional clásica no provee un DDL para definir una base de datos relacional, por ello es que para este trabajo se decidió definir un DDL para el álgebra relacional fuertemente inspirado en el DDL de SQL. Este DDL ofrece al usuario funcionalidades para:

- Crear una relación de forma atómica. Esto significa que el usuario deberá especificar el esquema de la relación.
- Crear una relación a partir de una consulta.
- Destruir una relación.
- Insertar tuplas dentro de una relación.
- Remover tuplas de una relación.

2.2. Consultas sobre una base de datos relacional

El DML (Data Manipulation Language) es un lenguaje que nos permite realizar consultas sobre una conjunto de relaciones/tablas definidas. Para poder realizar consultas sobre una base de datos relacional, el DML conocido como álgebra relacional, utiliza las operaciones fundamentales del álgebra relacional. De esta forma es que el DML ofrece al usuario los siguientes operadores para realizar consultas sobre una base de datos relacional:

- **select**. Esta operación filtra una relación según un predicado.
- **project**. Este operador permite seleccionar atributos específicos de una relación.
- **castesian product**. Esta operación realiza el producto cartesiano entre dos relaciones.
- **difference**. Este operador permite hallar las tuplas que están en una relación pero no en la otra.
- **union**. Esta operación permite combinar las tuplas de dos relaciones.
- **rename**. Este operador cambia el nombre de una relación.
- **renameA**. Esta operación permite cambiar el nombre de los atributos de una relación.

2.3. Definición de operadores personalizados

Las operaciones fundamentales del álgebra relacional son suficientes para expresar cualquier consulta del álgebra relacional. Sin embargo, limitándose exclusivamente a las operaciones fundamentales, algunas consultas habituales resultan complicadas de expresar. Por tanto, el EDSL ofrece al usuario una forma de:

- Definir sus propios operadores, permitiendo que se extienda el lenguaje.
- Des-definir operadores definidos, permitiendo que se comprima el lenguaje.

2.4. Sintaxis del EDSL

Antes de ver algunos ejemplos del EDSL, es necesario comprender la sintaxis del mismo.

Sintaxis concreta

La sintaxis concreta para el EDSL del álgebra relacional es la siguiente:

```

1  <edsl-statement> ::= (<ddl-command> | <dml-query> | <custom-op>) ";"
2
3  <ddl-command> ::= "CREATE" <rel-name> (<schema> | "AS" <dml-query>)
4                  | "DROP"   <rel-name>
5                  | "INSERT" <rel-name> "(" <attr-list> ")" "{" <tuple-list> "}"
6                  | "REMOVE" <rel-name> <predicate>
7
8  <dml-query> ::= <rel-name>
9              | "select"  "{" <predicate> "}"      "(" <dml-query> ")"
10             | "project" "{" <attr-list> "}"      "(" <dml-query> ")"
11             | "rename"  "{" <rel-name> "}"        "(" <dml-query> ")"
12             | "renameA" "{" <renamings-list> "}"   "(" <dml-query> ")"
13             | "union"    "(" <dml-query> "," <dml-query> ")"
14             | "difference" "(" <dml-query> "," <dml-query> ")"
15             | "cartesian_product" "(" <dml-query> "," <dml-query> ")"
16             | <op-name> "(" <query-list> ")"
17
18  <custom-op> ::= "DEFINE"   <op-name> "(" <arg-list> ")" "=" <dml-query>
19              | "UNDEFINE" <op-name>
20
21  <predicate> ::= <atom> <cmp-op> <atom>
22              | <predicate> "&&" <predicate>
23              | <predicate> "||" <predicate>
24              | "!" <predicate>
25              | "True" | "False"
26              | "(" <predicate> ")"
27
28  <atom> ::= <value> | <attr>
29
30  <cmp-op> ::= "==" | "!=" | "<" | "<=" | ">" | ">="

```

Sintaxis concreta del EDSL (BNF) - Parte 1

```
1  <schema> ::= "{" <attr-name> <domain> ("," <attr-name> <domain>)* "}"
2
3  <domain> ::= "INT" | "STRING" | "DOUBLE"
4
5
6  <value> ::= <string> | <int> | <double>
7
8  <string> ::= (<alpha-num>)*
9
10 <digit> ::= "0" | "1" | ... | "9"
11
12 <int> ::= "-" <natural> | <natural>
13
14 <natural> ::= <digit> | <digit> <natural>
15
16 <double> ::= <natural> "." <natural>
17
18
19 <attr-list> ::= <attr> ("," <attr>)*
20
21 <attr> ::= <rel-name> "." <attr-name>
22
23 <tuple-list> ::= <tuple> ("," <tuple>)*
24
25 <tuple> ::= "(" <value-list> ")"
26
27 <value-list> ::= <value> ("," <value>)*
28
29 <arg-list> ::= <arg-name> ("," <arg-name>)*
30
31 <queries-list> ::= <dml-query> ("," <dml-query>)*
32
33 <renamings-list> ::= <attr> "=" <attr> ("," <attr> "=" <attr>)*
34
35
36 <rel-name> ::= <identifier>
37
38 <op-name> ::= <identifier>
39
40 <arg-name> ::= <identifier>
41
42 <attr-name> ::= <identifier>
43
44 <identifier> ::= <letter> ( <alpha-num> | "_" )*
```

Sintaxis concreta del EDSL (BNF) - Parte 2

2.4.1. Sintaxis abstracta del DDL

La sintaxis abstracta para el EDSL del álgebra relacional es la siguiente:

```

1  -- Definicion de una sentencia del EDSL
2  data EDSLCommand = CreateRelation  RelationName (Either Relation Query)
3                    | DropRelation  RelationName
4                    | InsertTuple   RelationName [Tuple]
5                    | RemoveTuple   RelationName Predicate
6                    | DefineOperator OperatorName CustomOperator
7                    | UndefineOperator OperatorName
8                    | QueryStatement Query
9                    | EUnknown
10
11 -- Definicion de una consulta del algebra relacional
12 data Query = Atomic  RelationName
13            | Variable VariableName
14            | Rename  RelationName  Query
15            | Select  Predicate     Query
16            | Binary  BinaryOperator Query Query
17            | Project [AttributeKey] Query
18            | RenameA AttributeKeyMap Query
19            | Custom  OperatorName  [Query]
20
21 -- Definicion de un predicado del algebra relacional
22 data Predicate = PAnd  Predicate      Predicate
23              | POr   Predicate      Predicate
24              | PNot  Predicate
25              | PBool Bool
26              | PCmp  ComparisonOperator PredicateAtom PredicateAtom
27
28 -- Definicion de un atomo en predicados
29 data PredicateAtom = ConstantValue      Value
30                  | AttributeReference  RelationName AttributeName
31
32 -- Comparadores en predicados
33 data ComparisonOperator = Equal
34                       | NotEqual
35                       | LessThan
36                       | LessOrEqual
37                       | GreaterThan
38                       | GreaterOrEqual

```

Sintaxis abstracta del EDSL (Haskell) - Parte 1


```
1  -- Definicion de una relacion
2  data Relation = Relation (Maybe RelationName) Schema Instance RelationArity
3
4  -- Esquema de una relacion
5  type Schema    = [(AttributeKey, Domain)]
6
7  -- Instancia de una relacion
8  type Instance  = Set.Set Tuple
9
10 -- Aridad de una relacion
11 type RelationArity = Int
12
13 -- Definicion de una tupla
14 type Tuple      = Map.Map (RelationName, AttributeName) Value
15
16 -- Definicion de un valor
17 data Value =   IntValue      Int
18             |   StringValue String
19             |   DoubleValue Double
20
21 -- Definicion de un dominio
22 data Domain =   IntDomain
23             |   StringDomain
24             |   DoubleDomain
25
26 -- Operadores binarios en relaciones
27 data BinaryOperator = Union
28                   | Difference
29                   | CartesianProduct
30
31 -- Sinonimo para el tipo Map.Map AttributeKey AttributeKey
32 type AttributeKeyMap = Map.Map AttributeKey AttributeKey
33
34 -- Aridad de un operador
35 type OperatorArity = Int
36
37 -- Operadores custom o definidos por el usuario
38 data CustomOperator = CustomOperator OperatorArity [VariableName] Query
39
40 -- Sinonimos de tipos
41 type RelationName = String           -- Nombre de las relaciones
42 type OperatorName = String           -- Nombre de los operadores
43 type AttributeName = String           -- Nombre de los atributos
44 type VariableName = String           -- Nombre de las variables
45 type AttributeKey = (RelationName, AttributeName) -- Claves para el esquema
```

2.5. Ejemplos

Esta sección presenta ejemplos prácticos del lenguaje, organizados por nivel de complejidad. Cada ejemplo muestra tanto el código como el resultado esperado, facilitando la comprensión de las operaciones del álgebra relacional.

2.5.1. Uso del DDL

La forma más básica de crear una relación es definiendo su esquema:

```
1 CREATE estudiantes {nombre STRING, edad INT, ciudad STRING, promedio DOUBLE};
```

Ejemplo 1: Crear relación vacía

Esta sentencia crea una nueva relación llamada *estudiantes*, la cual almacena información de los estudiantes de una facultad. La relación *estudiantes* tiene 4 atributos: un atributo **nombre** de tipo STRING el cual almacena el nombre de un estudiante, un atributo llamado **ciudad** de tipo STRING el cual contiene la ciudad donde nació un estudiante, un campo **promedio** de tipo DOUBLE el cual guarda el promedio de un estudiante, y un atributo **edad** de tipo INT que almacena la edad de un estudiante. Esta relación inicialmente está vacía, por lo que vamos insertaremos algunas tuplas:

```
1 INSERT estudiantes (estudiantes.nombre,  
2                       estudiantes.edad,  
3                       estudiantes.ciudad,  
4                       estudiantes.promedio) {  
5     ("Ana", 20, "Rosario", 9.4),  
6     ("Bob", 22, "CABA", 8.1),  
7     ("Clara", 19, "Cordoba", 6.3),  
8     ("David", 53, "Rosario", 7.5),  
9     ("Elena", 18, "CABA", 10.0)  
10  };
```

Ejemplo 2: Insertar tuplas en una relación

Notemos que cuando insertamos tuplas, especificamos el orden de los atributos de la relación. En el ejemplo anterior, el primer elemento de la tupla representa al atributo **nombre**, el segundo elemento representa al atributo **edad**, el tercer elemento representa al atributo **ciudad** y el último elemento de la tupla representa el atributo **promedio**.

Ahora bien, podremos notar que cada atributo le precede el prefijo "estudiantes.". Esto se debe a que en este trabajo se decidió representar un atributo mediante un par ordenado (*relName*, *attrName*), donde:

- *relName* representa el nombre de la relación donde fue definido el atributo *attrName*. A partir cuando haga alusión al término *prefijo de relación*, voy estar hablando de *relName*.
- *attrName* representa el nombre del atributo. Por otro lado cuando hable del *nombre del atributo*, estoy hablando de *attrName*.

Esto nos permite que dentro de una relación puedan existir dos atributos con el mismo nombre siempre y cuando hayan sido definidos en relaciones diferentes. Esto resulta ampliamente útil cuando realizamos operaciones como **cartesian_product**.

El resultado de "Ejemplo 2: Insertar tuplas en una relación" produce que la tabla *estudiantes* tenga la siguiente instancia:

estudiantes.nombre	estudiantes.edad	estudiantes.ciudad	estudiantes.promedio
Ana	20	Rosario	9.4
Bob	22	CABA	9.1
Clara	19	Cordoba	6.3
David	53	Rosario	7.5
Elena	18	CABA	10.0

Observación sobre la representación de las tablas

A partir de este momento voy a dejar de escribir el prefijo *relName*. cada vez que muestro una tabla, a menos que valga la pena mostrarlos. De otra forma las tablas abarcarían un espacio mayor al que dispone las hojas del informe.

Por otro lado, también podríamos crear un relación a partir de una consulta. Por ejemplo podemos definir la relación *rosarinos* que contenga a todos los estudiantes rosarinos de la facultad:

```
1 CREATE rosarinos AS select{estudiantes.ciudad == "Rosario"}(estudiantes);
```

Ejemplo 3: Definir una relación a partir de una consulta

Esto produce la relación *rosarinos* tenga la siguiente instancia:

nombre	edad	ciudad	promedio
Ana	20	Rosario	9.4
David	25	Rosario	7.5

En un momento puede que se necesite eliminar elementos dentro de una relación. A continuación vemos como eliminar aquellos estudiantes mayores a 50 años:

```
1 REMOVE estudiantes estudiantes.edad > 50;
```

Ejemplo 4: Eliminar estudiantes mayores a 50 años

Como resultado tenemos que la relación *estudiantes* tiene la siguiente instancia:

nombre	edad	ciudad	promedio
Ana	20	Rosario	9.4
Bob	22	CABA	9.1
Clara	19	Cordoba	6.3
Elena	18	CABA	10.0

Para eliminar completamente una relación debemos utilizar la siguiente operación del DDL:

```
1 DROP estudiantes;
```

Ejemplo 5: Destruir una relación

Con esto finalizamos con los ejemplos del DDL.

2.5.2. Uso del DML

Para estos ejemplos vamos a considerar que dentro de la base de datos relacional existe relaciones *estudiantes*, *cursos* y *inscripciones* con las siguientes instancias:

nombre	edad	ciudad	codigo	nombre	creditos	estudiante	curso	nota
Ana	20	Rosario	MAT01	Matematica	6	Ana	MAT01	8
Bob	22	CABA	INF01	Informatica	4	Ana	INF01	9
Clara	19	Cordoba	FIS01	Fisica	5	Bob	MAT01	7
David	25	Rosario	QUI01	Quimica	4	Clara	FIS01	6
Elena	18	CABA				David	INF01	5
						Elena	MAT01	10

Primero comenzamos viendo como obtener de la base de datos, la información de los estudiantes mayores a 18 años:

```
1 select {estudiantes.edad > 18}(estudiantes);
```

Ejemplo 6: Operación **select** - Parte 1

Esta consulta arroja el siguiente resultado:

nombre	edad	ciudad
Ana	20	Rosario
Bob	22	CABA
Clara	19	Cordoba
David	25	Rosario

También podríamos realizar un filtrado más complejo si modificamos un poco el predicado. Por ejemplo, para obtener los estudiantes mayores a 18 años que viven en Rosario podríamos realizar la siguiente consulta:

```
1 select {estudiantes.edad > 18 && estudiantes.ciudad == "Rosario"}(estudiantes);
```

Ejemplo 6: Operación **select** - Parte 2

Esta consulta devuelve como resultado la siguiente relación:

nombre	edad	ciudad
Ana	20	Rosario
David	25	Rosario

El predicado dentro de la operación **select** no es más que una fórmula de lógica proposicional. Esto significa que podemos utilizar operadores lógicos como "y" (representado como un "&&"), "o" (representado como un "||"), "not" (representado como un "!"). En este caso las proposiciones son comparaciones entre atributos de relaciones y valores constantes.

A su vez notemos los atributos que aparezcan dentro del predicado deben especificarse mediante su prefijo de relación y el nombre del atributo.

Por otro lado, podemos quedarnos con atributos específicos de una relación utilizando el operador **project**. Por ejemplo podemos quedarnos con los nombres de los estudiantes de la siguiente forma:

```
1 project {estudiantes.nombre}(estudiantes);
```

Ejemplo 7: Operación **project** - Parte 1

Esta consulta produce este resultado:

```
+-----+
| nombre |
+-----+
| Ana    |
| Bob    |
| Clara  |
| David  |
| Elena  |
+-----+
```

Similar al operador **select**, los atributos que desean proyectar deben especificarse mediante el prefijo de relación y el nombre del atributo.

También podríamos quedarnos con las ciudades donde nació un alumno de la facultad:

```
1 project{estudiantes.ciudad}(estudiantes);
```

Ejemplo 7: Operación **project** - Parte 2

Esta consulta nos devuelve la siguiente instancia:

```
+-----+
| ciudad |
+-----+
| Rosario |
| CABA    |
| Cordoba |
+-----+
```

Notemos que, fiel al álgebra relacional, las relaciones no almacenan valores repetidos.

Si queremos combinar dos relaciones deberíamos utilizar el operador **union**. Por otra parte, si queremos quedarnos con todos los elementos de una relación que a su vez no pertenecen a otra, entonces deberíamos usar el operador **difference**. Para estos ejemplos vamos a considerar las siguientes relaciones creadas a partir de consultas:

```
1 CREATE estudiantes_rosario AS select{estudiante.ciudad == "Rosario"}(
   estudiantes);
2
3 CREATE estudiantes_cordoba AS select{estudiante.ciudad == "Cordoba"}(
   estudiantes);
```

Ejemplo 8: Operación **union** y **difference** - Parte 1

Estas relaciones tienen las siguientes instancias:

nombre	edad	ciudad
Ana	20	Rosario
David	25	Rosario

nombre	edad	ciudad
Clara	19	Cordoba

Si queremos obtener el conjunto de estudiantes que NO son rosarinos, entonces podemos realizar la siguiente consulta:

```
1 difference(estudiantes, estudiantes_rosario);
```

Ejemplo 8: Operación **union** y **difference** - Parte 2

Produciendo el resultado:

nombre	edad	ciudad
Bob	22	CABA
Clara	19	Cordoba
Elena	18	CABA

Por otro lado si queremos obtener el conjunto de estudiantes de Rosario y Córdoba, podemos realizar la consulta:

```
1 union(estudiantes_rosario, estudiantes_cordoba);
```

Ejemplo 8: Operación **union** y **difference** - Parte 3

Obteniendo así el resultado:

nombre	edad	ciudad
Ana	20	Rosario
Clara	19	Cordoba
David	25	Rosario

Un detalle importante de estos operadores es que solo puede realizarse entre relaciones que posean esquemas similares. Dos esquemas se dicen *similares* si la única diferencia entre los mismos son los identificadores de los atributos (los pares ordenados (*relName*, *attrName*)). De esta forma resulta que:

- Los esquemas [(*estudiantes.nombre*, STRING), (*estudiantes.edad*, INT)], [(*curso.nombre*, STRING), (*curso.cantidad_estudiantes*, INT)] son similares.
- Los esquemas [(*estudiantes.nombre*, STRING), (*estudiantes.edad*, INT)], [(*estudiantes.nombre*, STRING), (*estudiantes.edad*, STRING)] no son similares.
- Los esquemas [(*estudiantes.nombre*, STRING), (*estudiantes.edad*, INT)], [(*estudiantes.edad*, INT), (*estudiantes.nombre*, STRING)] no son similares.

Otra operación muy utilizada en el álgebra relacional es el **cartesian product**. Esta operación nos permite realizar el producto cartesiano entre dos instancias cualesquiera. Por ejemplo podríamos realizar producto cartesiano entre la relación **estudiantes** y la relación **cursos** a través de la siguiente consulta:

```
1 cartesian_product(estudiantes, cursos);
```

Ejemplo 9: Operación **cartesian product** - Parte 1

Esta consulta arroja el resultado:

nombre	edad	ciudad	codigo	nombre	creditos
Ana	20	Rosario	MAT01	Matematica	6
Ana	20	Rosario	INF01	Informatica	4
Ana	20	Rosario	FIS01	Fisica	5
Ana	20	Rosario	QUI01	Quimica	4
Bob	22	CABA	MAT01	Matematica	6
Bob	22	CABA	INF01	Informatica	4
Bob	22	CABA	FIS01	Fisica	5
Bob	22	CABA	QUI01	Quimica	4
Clara	19	Cordoba	MAT01	Matematica	6
Clara	19	Cordoba	INF01	Informatica	4
Clara	19	Cordoba	FIS01	Fisica	5
Clara	19	Cordoba	QUI01	Quimica	4
David	25	Rosario	MAT01	Matematica	6
David	25	Rosario	INF01	Informatica	4
David	25	Rosario	FIS01	Fisica	5
David	25	Rosario	QUI01	Quimica	4
Elena	18	CABA	MAT01	Matematica	6
Elena	18	CABA	INF01	Informatica	4
Elena	18	CABA	FIS01	Fisica	5
Elena	18	CABA	QUI01	Quimica	4

Fielmente al álgebra relacional clásica, el DML de este trabajo no permite realizar un producto cartesiano entre una misma relación. Es decir, la siguiente consulta es considerada ilegal:

```
1 cartesian_product(r, r);
```

Ejemplo 9: Operación **cartesian product** - Parte 2

Donde r es una relación cualquiera.

Esto se debe a que si realizamos el producto cartesiano entre r y r , entonces vamos a tener atributos repetidos (ya que los atributos tendrán el mismo nombre y provienen de la misma relación). Tener atributos repetidos hace que empiecen a aparecer problemas en los demás operadores. Consideremos el siguiente caso:

- Supongamos sin perder generalidad que se realiza la siguiente consulta sobre una relación r que posee el atributo $r.a$ repetido (suponemos además que *const* es un valor constante dentro del dominio de $r.a$):

```
1 select {r.a == const}(r);
```

Ejemplo 10: Problema con los atributos repetidos

- ¿Qué valor de $r.a$ se selecciona al momento de verificar el predicado de la operación **select**?
- Esto presenta un claro problema de ambigüedad ya que dependiendo de que valor de $r.a$ se tome puede variar el resultado.

Por esta razón es que se decidió restringir el producto cartesiano a relaciones distintas. Pero esta no es la única restricción que posee el producto cartesiano. Este trabajo no permite que se realice un producto cartesiano entre dos relaciones que tienen atributos en común (de ahí que no se pueda realizar un producto cartesiando con las mismas relaciones).

El lector podría argumentar entonces que en el "Ejemplo 9 - Parte 1" se presenta una consulta ilegal ya que las relación *estudiantes* y la relación *materias* comparten el atributo **nombre**. Debe recordarse que dichos atributos son distintos ya que provienen de relaciones distintas.

Estas restricciones terminan limitando la expresividad del lenguaje, por ello es que se implementan los operadores **rename** y **renameA**. El operador **rename** permite renombrar una relación atómica. Por otra parte, el operador **renameA** permite renombrar un conjunto de atributos de una relación.

Por ejemplo la siguiente consulta renombra a la relación *estudiantes* como *students*:

```
1 rename {students}(estudiantes);
```

Ejemplo 11: Operación **rename**

Ahora la relación *estudiantes* se llama *students*, y tiene la siguiente instancia:

students.nombre	students.edad	students.ciudad
Ana	20	Rosario
Bob	22	CABA
Clara	19	Cordoba
David	25	Rosario
Elena	18	CABA

Un detalle del operador **rename** es que solo genera cambios en relaciones atómicas, es decir, en relaciones definidas a partir de la sentencia **CREATE** que vimos en "Ejemplo 1: Crear relación vacía". Esto se debe a que renombrar el resultado de una consulta puede llevar a inconsistencias, por ejemplo si quisiéramos renombrar el resultado del "Ejemplo 9: Operación **cartesian product** - Parte 1" entonces caemos nuevamente en el mismo problema que tenía el producto cartesiano (el resultado posee atributos repetidos).

Por otro lado, la siguiente consulta cambia el nombre del atributo **nombre** a **nombre_estudiante**:

```
1 renameA{estudiantes.nombre = estudiantes.nombre_estudiante}(estudiantes);
```

Ejemplo 12: Operación **renameA**

El resultado de dicha consulta es el siguiente:

nombre_estudiantes	edad	ciudad
Ana	20	Rosario
Bob	22	CABA
Clara	19	Cordoba
David	25	Rosario
Elena	18	CABA

Notemos que **renameA** también nos permite cambiar el prefijo de relación de un atributo.

2.5.3. Definición de operadores personalizados

Para crear operadores personalizados tenemos la sentencia **DEFINE**. Por ejemplo podemos definir el operador intersección de la siguiente manera:

```
1 DEFINE intersection(r, s) = difference(r, difference(r, s));
```

Ejemplo 13: Operación **DEFINE**

Luego podemos usar este operador dentro de otras consultas tal y como lo haríamos con cualquier otro operador. Por ejemplo la siguiente consulta calcula la intersección entre los estudiantes rosarinos y los estudiantes mayores de 23 años:

```
1 intersection(select{estudiantes.ciudad == "Rosario"}(estudiantes), select{
    estudiantes.edad > 23}(estudiantes));
```

Ejemplo 14: Ejemplo de uso de un operador personalizado

El resultado de esta consulta sería el siguiente:

+	-----+	-----+	-----+
	nombre		edad
			ciudad
+	-----+	-----+	-----+
	David		25
			Rosario
+	-----+	-----+	-----+

Si quisiéramos destruir el operador intersección deberíamos usar la sentencia **UNDEFINE** de la siguiente manera:

```
1 UNDEFINE intersection;
```

Ejemplo 15: Operación **UNDEFINE**

De esta forma el usuario ya no puede utilizar el operador 'interacción'.

2.6. Detalles del diseño del EDSL

Para finalizar con la esta parte vamos a ver algunos detalles del diseño de este EDSL.

- La sentencia **CREATE** sobrescribe relaciones existentes. Es decir si se define una relación **r**, y luego se vuelve a definir, entonces está última definición de **r** es la que prevalece.
- Las relaciones definidas a través de la sentencia **CREATE ... AS** se consideran relaciones no atómicas.
- La sentencia **DEFINE** sobrescribe operadores existentes.
- Este trabajo restringe los dominios que puede poseer un atributo. En particular soporta los dominios: INT, STRING y DOUBLE.

- Este trabajo no soporta valores nulos. Es decir, las tuplas deben tomar un valor no nulo dentro de cada atributo. Tomé esta decisión con el objetivo de no complejizar las operaciones.
- El resultado de una consulta se considera una relación no atómica (de aquí que las relaciones definidas con **CREATE ... AS** sean no atómicas). Tomé esta decisión para que los renombramientos de relaciones y renombramientos de atributos no produzcan estados ilegales.
- Los predicados permiten que se escriban comparaciones entre atributos de diferentes dominios, por ejemplo el predicado:

estudiantes.nombre == estudiantes.edad

Es válido, más allá de que el atributo **estudiantes.nombre** tenga dominio **STRING** y el atributo **estudiantes.edad** tenga dominio **INT**. En estos casos, la comparación devuelve el valor **False**

- Los atributos nombrados dentro de los sub-índices de los operadores **select**, **project** y **renameA** deben pertenecer a la relación sobre la que se aplica el operador. De otra forma la consulta se considera ilegal.

3 Implementación del EDSL

Para cerrar con este informe voy a hablar sobre la implementación del EDSL.

3.1. Módulos del trabajo

El trabajo práctico contiene 5 cinco módulos principales, cada uno con una responsabilidad específica. A continuación se describe brevemente el propósito de cada módulo, para luego analizarlos en detalle:

- **Parser.hs**: En este módulo se encuentra el parser de este trabajo. Ofrece al intérprete el parser *parserStatements* con el cual puede parsear sentencias del EDSL.
- **Common.hs**: Dentro de este módulo se definen todos los tipos de datos algebraicos compartidos por el resto del sistema. Incluye la representación del AST (comandos y consultas), la estructura de relaciones, predicados, valores y el estado del intérprete. Este módulo no contiene lógica de procesamiento, solo definiciones de tipos.
- **Eval.hs**: En este módulo se define el evaluador del DML. Se encarga de implementar la semántica de las operaciones de consulta del álgebra relacional. Además, maneja el estado del sistema y el sistema de errores de lenguaje de consultas.
- **PrettyPrinter.hs**: Este módulo es el encargado del formateo de resultados y mensajes de error. Transforma relaciones en tablas ASCII legibles y errores en mensajes informativos.
- **Main.hs**: Es el módulo que se encarga de implementar el intérprete interactivo del EDSL. Se encarga principalmente de la interacción con el usuario. Utiliza los otros 4 módulos para implementar sus funcionalidades.

3.1.1. Módulo Parser.hs

El módulo **Parser.hs** es el encargado principal de implementar el parser de este trabajo. Para poder implementarlo se utilizó la librería Parsec ya que fue utilizada en trabajos previos de la materia. La otra opción era utilizar la librería Happy, pero consideré que el uso de esta última librería resultaba innecesariamente complejo para este trabajo.

La función principal de este módulo es *parserStatements*. Esta función permite parsear un conjunto de sentencias del EDSL, devolviendo como resultado un conjunto de comandos. Formalmente el tipo de esta función es:

```
1 parserStatements :: SourceName -> String -> ParseResult [(EDSLCommand, Int)]
```

Tipo de la función *parserStatements* (Haskell)

La definición del tipo ParseResult (la cual se encuentra dentro del módulo **Common.hs**) es la siguiente:

```
1 data ParseResult a = Failed String | Ok a
```

Definición del tipo ParseResult (Haskell)

De esta forma:

- Si ocurre un error al momento de parsear, la función ***parserStatements*** devuelve Failed str. El string str especifica el error de parseo que ocurrió, y la línea y columna donde ocurrió el error.
- Si el parseo se da de forma exitosa, entonces la función ***parserStatements*** devuelve Ok cs. Donde cs es una lista con elementos de tipo (EDSLCommand, Int). El primer elemento de uno de estos pares ordenados representa un comando del EDSL y el segundo elemento representa la línea donde se encuentra dicho comando dentro de un archivo.

3.1.2. Módulo Eval.hs

El módulo Eval.hs implementa el evaluador de este trabajo. Este último se encarga de implementar la semántica del álgebra relacional y también se encarga del manejo de errores. Las funciones que exporta este módulo son:

- ***evalQuery***. Esta función recibe el estado del intérprete, una consulta del álgebra relacional, y realiza la consulta sobre la base de datos definida. Si la consulta pudo realizarse sin problemas, entonces la función devuelve la relación resultante (es decir, el resultado de la consulta). Por otro lado, si la consulta del usuario presenta algún problema al ejecutarse, entonces la función devuelve un error contextual el cual permitirá al usuario ver el problema que presenta su consulta.
- ***evalPredicate***. Esta función recibe una relación, el AST de un predicado, y devuelve un predicado en Haskell equivalente al predicado ingresado por AST. Este predicado toma una tupla y devuelve un booleano. Si esta transformación del AST al predicado en Haskell presenta un problema, entonces la función ***evalPredicate*** devuelve un error de evaluación que explica el problema que se encontró al realizar la evaluación.

Para comprender en su totalidad las funciones de este módulo hace falta explicar algunas cosas. Primero comenzaremos el tipo de la función ***evalQuery***:

```
1 evalQuery :: State -> Query -> EvalResult Relation
```

Tipo de la función ***evalQuery*** (Haskell)

Analicemos cada uno de estos tipos.

El tipo 'State'

El tipo 'State' representa el estado del intérprete. Contiene información sobre las relaciones creadas por el usuario, operadores custom definidos por el usuario, y el nombre del último archivo cargado por el usuario. Formalmente el tipo 'State' se define como:

```
1 data State = S {lfile :: FileName,
2                rs     :: RelationsEnvironment,
3                ops     :: OperatorsEnvironment}
```

Definición del tipo State (Haskell)

Donde:

- *lfile* almacena el nombre del último archivo cargado por el usuario. El tipo 'FileName' se define como:

```
1 type FileName = String
```

Definición del tipo FileName (Haskell)

- *rs* contiene todas las relaciones creadas por el usuario (siempre que no hayan sido destruidas). El tipo 'RelationsEnvironment' se define como:

```
1 type RelationsEnvironment = Map.Map RelationName Relation
```

Definición del tipo RelationsEnvironment (Haskell)

- *ops* guarda todos los operadores custom definidos por el usuario (siempre que no hayan sido des-definidos). El tipo 'OperatorsEnvironment' se define como:

```
1 type OperatorsEnvironment = Map.Map OperatorName CustomOperator
```

Definición del tipo OperatorsEnvironment (Haskell)

Los tipos 'RelationName', 'RelationName', 'OperatorName' y 'CustomOperator' ya fueron definidos cuando vimos la sintaxis abstracta del lenguaje.

El campo *lfile* no es utilizado por el evaluador, su uso aparece dentro de la implementación de uno de los comandos del intérprete. Por otra parte, los campos *rs* y *ops* son muy importantes a la hora de evaluar consultas ya que:

- El campo *rs* permite al evaluador buscar relaciones referenciadas en las consultas.
- El campo *ops* permite al evaluador buscar definiciones de operadores personalizados.

El tipo 'EvalResult'

El tipo 'EvalResult' es un tipo de datos definido para el manejo de errores contextuales. Su definición es la siguiente:

```
1 type EvalResult a = Either ContextualError a
```

Definición del tipo EvalResult (Haskell)

De esta forma, si la función *evalQuery* no tiene ningún problema al evaluar una consulta entonces devuelve 'Right r' (donde 'r' es la relación resultante de la consulta). Por otra parte, si se presenta un error al realizar la consulta entonces la función *evalQuery* devuelve 'Left e' (donde 'e' es el error contextual que ocurrió al evaluar la consulta).

La definición del tipo 'ContextualError' es la siguiente:

```
1 data ContextualError = ContextualError Query ErrorContext EvalError
```

Definición del tipo ContextualError (Haskell)

Un 'ContextualError' no solo almacena el tipo de error que ocurrió (el cual se almacena dentro de la expresión de tipo 'EvalError'), también almacena la consulta que presentó dicho error (la cual puede ser la consulta que recibió la función *evalQuery*, o bien puede ser una sub-consulta dentro de la consulta que recibió la función *evalQuery*), y el contexto donde ocurrió el problema.

Para terminar de comprender esto es conveniente ver la definición de 'ErrorContext' y 'EvalError':

```
1 data EvalError = UndefinedRelation      RelationName
2                | UndefinedAttribute    AttributeKey
3                | UndefinedOperator     OperatorName
4                | ArityMismatch         OperatorName Int      Int
5                | UnboundVariable      VariableName
6                | IncompatibleSchemas
7                | SameRelationName     RelationName
8                | SharedAttributes     AttributeKey
9                | DuplicateAttributeName AttributeKey
10
11 data ErrorContext = InQuery
12                  | InPredicate        Predicate
13                  | InRenameAttributes
14                  | InProyect
15                  | InSubstitution
```

Definición del tipo EvalError y ErrorContext (Haskell)

El tipo 'EvalError' representa todos los posibles errores que pueden ocurrir durante la evaluación. Cada constructor representa un tipo específico de error:

- **UndefinedRelation:** Un error de este tipo ocurre cuando se intenta usar una relación que no existe dentro del estado actual del intérprete. Este tipo de error almacena el nombre de la relación que produjo el fallo.
- **UndefinedAttribute:** Este error ocurre cuando se utiliza un atributo que no pertenece a la relación. Este tipo de error almacena la clave del atributo que produjo el fallo.
- **UndefinedOperator:** Un error de este tipo ocurre cuando se intenta usar un operador personalizado que no existe dentro del estado actual del intérprete. Este tipo de error almacena el nombre de la operador personalizado que produjo el fallo.
- **ArityMismatch:** Este error se produce cuando la cantidad de argumentos de un operador personalizado no coincide con la cantidad de argumentos para los cuales fue definido. Este tipo de error almacena el nombre del operador personalizado, la cantidad de argumentos esperada y la cantidad de argumentos recibida.

- **UnboundVariable:** Un error de este tipo se produce cuando una variable aparece libre dentro de una consulta. Esta clase de error almacena el nombre de la variable que produjo el fallo.
- **IncompatibleSchemas:** Este error ocurre cuando se intenta realizar una operación de unión o diferencia entre dos relaciones con esquemas incompatibles.
- **SameRelationName:** Un error de esta clase ocurre cuando se intenta realizar un producto cartesiano entre dos relaciones con el mismo nombre. Este error almacena el nombre de la relación que produjo el fallo.
- **SharedAttributes:** Este error se produce cuando dos relaciones en un producto cartesiano comparten un atributo. Este tipo de error almacena la clave del atributo que produjo el fallo.

El tipo 'ErrorContext' indica en qué parte del programa ocurrió el error. Cada constructor indica dónde ocurrió el problema:

- **InQuery:** El error ocurrió dentro de una consulta.
- **InPredicate:** El error ocurrió dentro de un predicado.
- **InRenameAttributes:** El error ocurrió dentro de una operación de renombramiento de atributos.
- **InProject:** El error ocurrió dentro de una operación de proyección.
- **InSubstitution:** El error ocurrió al momento de sustituir los argumentos de un operador personalizado.

De esta forma el usuario obtiene mucha más información sobre el error que se produjo al querer evaluar su consulta.

Observación sobre consultas que poseen más de un error

El evaluador solo muestra el primer error que encuentra. Esto implica que si una consulta tiene más de un error, entonces el usuario deberá corregir un error, evaluar la consulta nuevamente, y así sucesivamente hasta que no queden errores.

3.1.3. Módulo Common.hs

El módulo **Common.hs** define todos los tipos de datos algebraicos del trabajo. Este módulo solo contiene definiciones de tipos, por ende no vamos a encontrar definiciones de funciones. Los tipos de datos principales que se definen en este trabajo son:

- **Resultados de parsear:** Se define el tipo de dato 'ParseResult' para representar los resultados de parsear una sentencia del EDSL.
- **Resultados de evaluar:** Definimos los tipos 'EvalResult', 'EvalError', 'ErrorContext' y 'ContextualError' para representar los resultados de evaluar una consulta.
- **Entorno:** Se definen los tipos 'State', 'RelationsEnvironment' y 'OperatorsEnvironment' para representar el estado del intérprete.

- **Comandos del EDSL:** Definimos el tipo de dato 'EDSLCommand' para representar los comandos del EDSL.
- **Comandos del intérprete:** Definimos los tipos de datos 'ICommand' y 'FileName' para representar los comandos del intérprete.
- **Relaciones:** Se definen los tipos de datos 'Relation', 'Instance', 'Schema' y 'RelationName' para representar las relaciones del álgebra relacional.
- **Consultas:** Se definen los tipos de datos 'Query', 'Predicate', 'AttributeKeyMap', 'ComparisonOperator', 'PredicateAtom' y 'BinaryOperator' para representar las consultas del álgebra relacional.
- **Operadores:** Se definen los tipos de datos 'Operator', 'OperatorName' y 'OperatorArity' para representar los operadores del álgebra relacional.
- **Valores:** Se define el tipo de dato 'Value' para representar los valores de las relaciones.
- **Atributos:** Se define los tipos de datos 'AttributeKey' y 'AttributeName' para representar los atributos de las relaciones.
- **Dominios:** Se define el tipo de dato 'Domain' para representar los dominios de las relaciones.

Muchos de estos tipos fui abarcandolos en otras secciones, por lo solo voy a detenerme a explicar los que no fueron explicados previamente.

Definición de una relación

Como bien se puede observar en el código, una relación se compone de:

- Un campo de tipo 'Maybe RelationName' el cual representa el nombre de una relación. Este campo es de tipo 'Maybe' ya que es las relaciones que se crean a partir de una consulta no tienen un nombre. En dicho caso, el nombre de esas relaciones será Nothing.
- Un campo de tipo 'Schema' el cual representa el esquema de una relación. Este campo tiene el siguiente tipo:

```
1 type Schema = [(AttributeKey, Domain)]
```

Definición del tipo Schema (Haskell)

- Un campo de tipo 'Instance' que representa la instancia de la relación. Este campo tiene el siguiente tipo:

```
1 type Instance = Set.Set Tuple
```

Definición del tipo Instance (Haskell)

Decidí representar una instancia con un conjunto ya que de esta forma es mucho más sencillo mantener la propiedad de que cada tupla solo puede aparecer una única vez en una relación (ya que de hecho las relaciones no son más que conjuntos). El tipo de 'Tuple' es el siguiente:

```
1 type Tuple = Map.Map (RelationName, AttributeName) Value
```

Definición del tipo Tuple (Haskell)

- Un campo 'RelationArity' que representa la aridad de la relación. Este campo tiene el siguiente tipo:

```
1 type RelationArity = Int
```

Definición del tipo RelationArity (Haskell)

Comandos del intérprete

El tipo 'ICommand' representa los comandos que puede ejecutar el intérprete. Los comandos están precedidos por el prefijo ':', y los distintos comandos que posee el intérprete son:

- **help/?**: Muestra por pantalla un mensaje al usuario con todos los comandos disponibles.
- **load**: Carga un archivo del álgebra relacional dentro en el intérprete. Los archivos del álgebra relacional son archivos con extensión '.ra' que contienen sentencias del EDSL. Puede que algunas de estas sentencias tengan errores, en dicho caso estas sentencias fallarán al momento de ser evaluadas. El fallo de una sentencia no afecta directamente a las sentencias que se encuentran antes o después de la sentencia que falló (a menos que estén relacionadas).
- **reload**: Recarga el último archivo del álgebra relacional que se cargó en el intérprete.
- **list_relations**: Imprime por pantalla todas las relaciones que se encuentran cargadas en el intérprete.
- **list_operators**: Imprime por pantalla todos los operadores personalizados que se encuentran cargados en el intérprete.
- **print_relation**: Imprime por pantalla una relación.
- **print_operator**: Imprime por pantalla un operador personalizado.
- **describe_relation**: Imprime por pantalla el esquema de una relación.
- **quit**: Termina la ejecución del intérprete.

Para ver con más detalle estos comandos se recomienda leer el readme del trabajo.

3.1.4. Módulo Main.hs

El módulo **Main.hs** implementa el intérprete que permite al usuario interactuar con el sistema. Veamos brevemente el funcionamiento del mismo:

- El intérprete comienza mostrando un mensaje de bienvenida, y seguidamente espera a que el usuario ingrese una entrada por la consola.
- Una vez que el intérprete recibe la entrada del usuario y llama a la función *interpretInput* con la entrada recibida. Esta función se fija si la entrada comienza o no con el prefijo ':'. En caso afirmativo llama a la función *interpretCommand*, en caso negativo llama a la función *interpretQuery*. Los tipos de estas funciones son:

```
1      interpretInput :: String -> InputT IO ICommand
2      interpretQuery :: String -> InputT IO ICommand
3      interpretCommand :: String -> InputT IO ICommand
```

Tipos de las funciones *interpretInput* (Haskell)

Notemos que tanto la función *interpretCommand* como *interpretQuery* devuelven un comando del intérprete.

- Luego el intérprete toma el comando resultante y lo maneja con la función *handleCommand*. Esta función recibe el estado del intérprete y un comando, y ejecuta dicho comando con el estado actual del intérprete. Esta función devuelve como resultado un 'Maybe State' con el nuevo estado del intérprete. El tipo de esta función es el siguiente:

```
1      handleCommand :: State -> ICommand -> InputT IO (Maybe State)
```

Tipos de la función *handleCommand* (Haskell)

- El intérprete vuelve a ejecutarse con el nuevo estado (en caso de que el resultado de *handleCommand* sea 'Nothing', el intérprete se cierra. Esto solo ocurre si el comando ingresado por el usuario fue ':quit').

3.1.5. Módulo PrettyPrinter.hs

El módulo **PrettyPrinter.hs** contiene las funciones encargadas de formatear elementos dentro de este trabajo. Para poder implementar las funciones de este módulo se utilizó la librería **HughesPJ** ya que es la que se utilizó en todos los trabajos previos de la materia.

Las funciones principales que exporta este módulo son:

- *ppRelation*. Esta función se encarga de formatear una relación a una tabla ASCII de forma que sean mucho más legibles para el usuario.
- *ppQuery*. Esta función convierte el AST de una consulta a un formato de texto.
- *ppOperator*. Esta función formatea la definición de un operador custom a un formato de texto.
- *ppSchema*. Esta función muestra el esquema de una relación en un formato legible.
- *ppError*. Esta función formatea los errores contextuales de forma que el usuario pueda ser informado de errores al momento de realizar una consulta.